Experiment No 7
LAB MANUAL
PART B

Roll No. C-60	**Name: Avdhut Parish Madhavi**
Class: SE-C	**Batch: C2**
Date of Experiment:	**Date of Submission:**
Grade:	

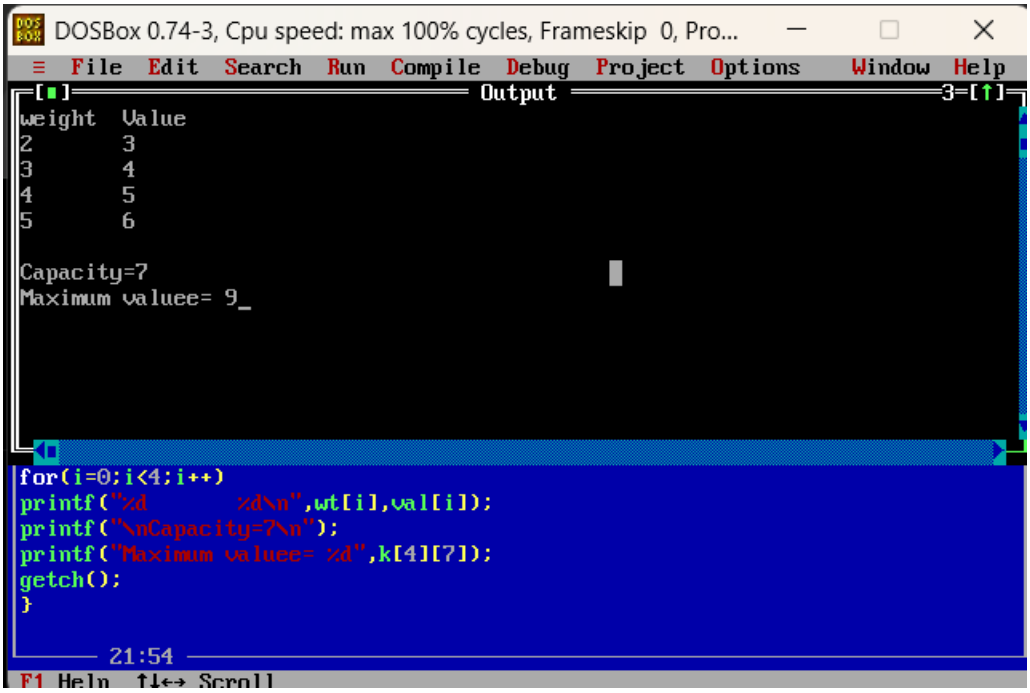
B.1: Implement and trace the following programs. Write the code, compile/run it, and record the output after each task.

- Run the program with a different set or order of items and verify the maximum value obtained using the Dynamic Programming table.

  CODE:

```
#include <stdio.h>
#include <conio.h>
int max(int a, int b)
{
   if(a > b)
      return a;
   return b;
}
void main()
{
   int wt[4] = {2, 3, 4, 5};
   int val[4] = {3, 4, 5, 6};
   int K[5][8];
   int i, w;
   clrscr();
   for(i = 0; i <= 4; i++)
   {
      for(w = 0; w <= 7; w++)
      {
         if(i == 0 || w == 0)
            K[i][w] = 0;
         else if(wt[i-1] <= w)
```

```
            K[i][w] = max(
               val[i-1] + K[i-1][w-wt[i-1]],
               K[i-1][w]);
         else
            K[i][w] = K[i-1][w];
      }
   }
   printf("Weight  Value\n");
   for(i = 0; i < 4; i++)
      printf("%d       %d\n", wt[i], val[i]);
   printf("\nCapacity = 7\n");
   printf("Maximum value = %d", K[4][7]);
   getch();
}
```



- Do the analysis of the 0/1 knapsack problem using Dynamic Programming.

CODE:

```
#include <stdio.h>
#include <conio.h>
void main()
{
```

```
    int n, W, i, j, cells;
    clrscr();
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter knapsack capacity: ");
    scanf("%d", &W);
    cells = (n + 1) * (W + 1);
    printf("\nDP table size = %d x %d", n+1, W+1);
    printf("\nTotal table cells = %d", cells);
    printf("\nTime Complexity = O(nW)");
    printf("\nSpace Complexity = O(nW)");
    printf("\n\nAnalysis:");
    printf("\nEach table cell is calculated once.");
    printf("\nHence time complexity is O(nW).");
    printf("\nThe table requires O(nW) space.");
    getch();
}
```

